Churn Prediction Model

Predicting At-Risk Learners on an Online Course Platform

Internship Project, internee.pk | Tools: Python, pandas, scikit-learn, matplotlib, seaborn

Objective

Predict which users are likely to leave (churn from) an online learning platform using engagement behaviour data, so at-risk users can be identified early and offered personalised support.

Dataset & Approach

500 users from `online_learning_course_consumption_dataset.csv` with hours spent per week, course duration, completion percentage, satisfaction score, experience level, course type and platform.

- Churn label from **Completion_Status**: **Dropped** = churned (1), all else = not churned (0).
- Removed **User_ID** (identifier) and **Dropout_Reason** (would leak the outcome).
- One-hot encoded Experience_Level, Course_Type, Platform; 80/20 stratified train/test split.
- Compared **Random Forest** and **Gradient Boosting** on precision, recall and F1 for the churn class.

1. Load & Explore the Data

IN

```
import pandas as pd

df = pd.read_csv("online_learning_course_consumption_dataset.csv")
df.head()
```

User_ID	Experience_ Level	Course_ Type	Platform	Hours_Spent_ Per_Week	Course_ Duration_Weeks	Completion_ Status	Completion_ Percentage	Dropout_ Reason	Satisfaction_ Score
U0001	Fresher	Tech	Skillshare	16	8	Completed	67	No Dropout	2
U0002	Fresher	Non-Tech	edX	17	22	Dropped	38	Time Constraint	3
U0003	Fresher	Non-Tech	Coursera	19	21	In Progress	43	No Dropout	1
U0004	Student	Non-Tech	Udemy	14	2	Dropped	14	Time Constraint	1
U0005	Student	Non-Tech	Skillshare	8	10	Dropped	8	Too Difficult	4

IN

```
df.info()
df.shape
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 10 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   User_ID                500 non-null    object
 1   Experience_Level       500 non-null    object
 2   Course_Type            500 non-null    object
 3   Platform               500 non-null    object
 4   Hours_Spent_Per_Week   500 non-null    int64
 5   Course_Duration_Weeks  500 non-null    int64
 6   Completion_Status      500 non-null    object
 7   Completion_Percentage  500 non-null    int64
 8   Dropout_Reason         500 non-null    object
 9   Satisfaction_Score     500 non-null    int64
dtypes: int64(4), object(6)

(500, 10)
```

2. Create the Churn Label & Encode Features

IN

```
# Create the churn label
df['Churn'] = df['Completion_Status'].apply(lambda x: 1 if x == 'Dropped' else 0)

# Drop identifier and target-leaking columns
features = df.drop(columns=['User_ID', 'Completion_Status', 'Dropout_Reason', 'Churn'])
target = df['Churn']

# One-hot encode categorical columns
features_encoded = pd.get_dummies(features, columns=['Experience_Level', 'Course_Type', 'Platform'],
                                  drop_first=True)
features_encoded.head()
```

Hours_ Spent_ Per_Week	Course_ Duration_ Weeks	Completion_ Percentage	Satisfaction_ Score	Experience_ Level_ Student	Experience_ Level_Working Professional	Course_ Type_ Tech	Platform_ Skillshare	Platform_ Udemy	Platform_ YouTube	Platform_ edX
16	8	67	2	False	False	True	True	False	False	False
17	22	38	3	False	False	False	False	False	False	True
19	21	43	1	False	False	False	False	False	False	False
14	2	14	1	True	False	False	False	True	False	False
8	10	8	4	True	False	False	True	False	False	False

3. Train & Evaluate Random Forest

IN

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(
    features_encoded, target, test_size=0.2, random_state=42, stratify=target
)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
```

IN

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

y_pred = rf_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

OUTPUT

```
Accuracy: 0.92

              precision    recall  f1-score   support

           0       0.98      0.89      0.94        65
           1       0.83      0.97      0.89        35

    accuracy                           0.92       100
   macro avg       0.91      0.93      0.92       100
weighted avg       0.93      0.92      0.92       100

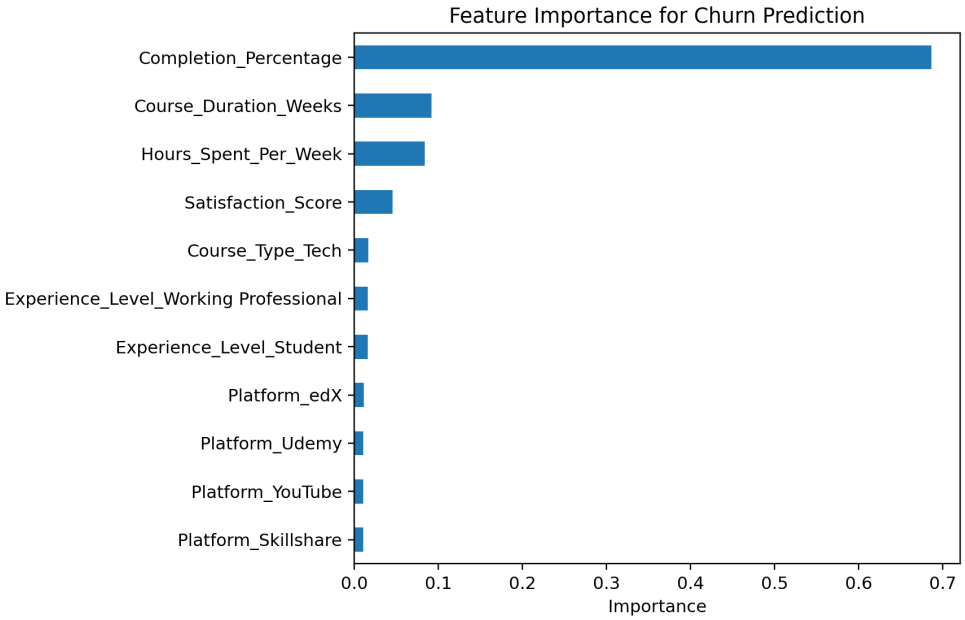
[[58  7]
 [ 1 34]]
```

4. Feature Importance

IN

```
importances = pd.Series(rf_model.feature_importances_, index=X_train.columns)

importances.sort_values().plot(kind='barh', figsize=(8,6), title="Feature Importance for Churn Prediction")
```



5. Train & Evaluate Gradient Boosting

IN

```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(random_state=42)
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred_gb))
print(classification_report(y_test, y_pred_gb))
```

OUTPUT

```
Accuracy: 0.92
              precision    recall  f1-score   support

           0       0.94      0.94      0.94        65
           1       0.89      0.89      0.89        35

    accuracy                           0.92       100
   macro avg       0.91      0.91      0.91       100
weighted avg       0.92      0.92      0.92       100
```

6. Compare the Two Models

IN

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

comparison = pd.DataFrame({
    "Model": ["Random Forest", "Gradient Boosting"],
    "Accuracy": [accuracy_score(y_test, y_pred_rf), accuracy_score(y_test, y_pred_gb)],
    "Precision (Churn)": [precision_score(y_test, y_pred_rf), precision_score(y_test, y_pred_gb)],
    "Recall (Churn)": [recall_score(y_test, y_pred_rf), recall_score(y_test, y_pred_gb)],
    "F1-score (Churn)": [f1_score(y_test, y_pred_rf), f1_score(y_test, y_pred_gb)]
})
```

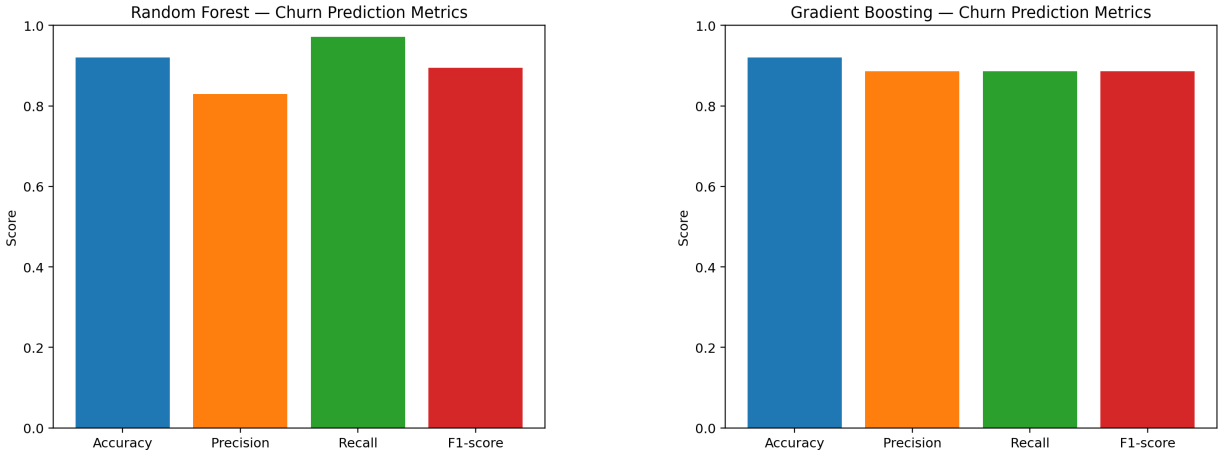
Model	Accuracy	Precision (Churn)	Recall (Churn)	F1-score (Churn)
Random Forest	0.92	0.829268	0.971429	0.894737

Gradient Boosting	0.92	0.885714	0.885714	0.885714

7. Per-Model Metrics

IN

```
plt.bar(["Accuracy", "Precision", "Recall", "F1-score"],
        [accuracy_score(y_test, y_pred_rf), precision_score(y_test, y_pred_rf),
         recall_score(y_test, y_pred_rf), f1_score(y_test, y_pred_rf)])
plt.title("Random Forest — Churn Prediction Metrics"); plt.ylim(0, 1)
```



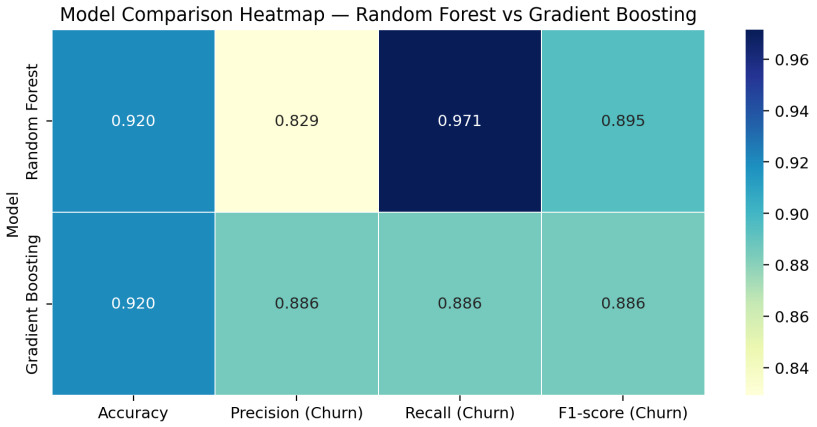
Random Forest (left) vs Gradient Boosting (right)

8. Heatmap Comparison

IN

```
import seaborn as sns

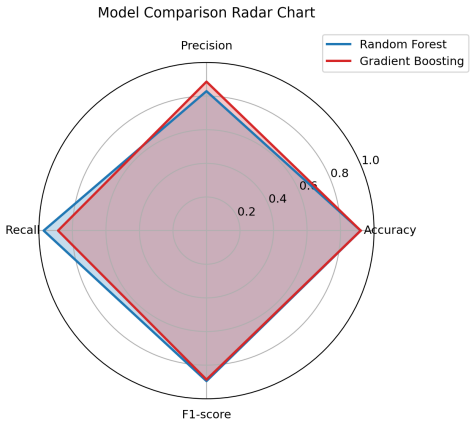
comparison_metrics = comparison.set_index("Model")[
    ["Accuracy", "Precision (Churn)", "Recall (Churn)", "F1-score (Churn)"]]
sns.heatmap(comparison_metrics, annot=True, fmt=".3f", cmap="YlGnBu", cbar=True, linewidths=0.5)
```



9. Radar Chart Comparison

IN

```
angles = np.linspace(0, 2*np.pi, len(metrics), endpoint=False).tolist()
ax = plt.subplot(polar=True)
ax.plot(angles, rf_scores, label="Random Forest"); ax.fill(angles, rf_scores, alpha=0.25)
ax.plot(angles, gb_scores, label="Gradient Boosting"); ax.fill(angles, gb_scores, alpha=0.25)
```



Summary & Recommendations

Key Insight

Completion_Percentage was by far the strongest predictor of churn (68.6% importance), well ahead of course duration, weekly hours and satisfaction score. An early drop in course progress is the clearest warning sign of churn.

Final Model Choice

Random Forest was selected. Both models tied on accuracy (0.92) and F1 (0.89), but Random Forest reached a recall of **0.97 vs 0.89** on churned users, missing far fewer at-risk learners. In churn prevention, missing a real churner costs more than a false alarm.

Recommendation

Monitor **Completion_Percentage** as an early-warning signal and trigger personalised support (reminders, check-ins, simplified content) as soon as a learner's progress stalls below the expected pace, instead of waiting for low satisfaction or inactivity.